The Little Book of Deep Learning: an explainer

This page explains passages of François Fleuret's *The Little Book of Deep Learning* (version 1.3, 2026) in more detail than the book has room for. The book is short and precise: a single page can hold an equation that takes a few paragraphs to unpack. Each section below takes one such passage, works through it with numbers, and then adds what the book leaves out under the heading "Beyond the book". References such as §3.1 point to the book's sections.

The book is free under the Creative Commons BY-NC-SA 4.0 licence. Its official page has the current version; an unmodified copy of version 1.3 is in the download box above.

The page grows as I read further. So far it covers the losses of §3.1 and backpropagation (§3.4), and ends with a section the book does not have: what a Jacobian is and how backpropagation is a product of them.

The numbers on this page were recomputed by a short numpy script, offered in the download box, and the gradients of sections 3 and 6 were checked against finite differences.

1. The exponential function

Softmax (section 2) is built from exp, so it is worth being precise about it first.

$\exp(x)$ is e^x: the constant $e \approx 2.71828$ raised to the power x. The book writes $\exp f(x;w)_y$ rather than $e^{f(x;w)_y}$; it is the same function, and the exp form stays readable when the exponent is a long expression.

The definition that works for every real x, not only whole-number powers, is a series:

$$\exp(x) = \sum_{k=0}^{\infty} \frac{x^k}{k!} = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots$$

Equivalently, exp is the only function that equals its own derivative and has $\exp(0) = 1$.

A few values for intuition: $\exp(0) = 1$, $\exp(1) \approx 2.72$, $\exp(2) \approx 7.39$, $\exp(-1) \approx 0.37$, $\exp(10) \approx 22\,026$. It grows very fast. That is why a modest lead in logits (the model's raw scores, section 2) becomes a dominant probability, and why naive implementations overflow on large inputs.

Five properties do all the work in what follows:

- **Always positive.** Any real input, even -1000, gives an output above 0. This is what lets arbitrary scores become valid probabilities.
- **Strictly increasing.** A larger input always gives a larger output, so the ranking of classes is preserved.
- **Turns addition into multiplication:** $\exp(a + b) = \exp(a)\exp(b)$. This is why adding a constant to all logits cancels out in softmax, and why logit differences become probability ratios.
- **Inverse of the natural logarithm:** $\log(\exp(x)) = x$. This is why the log in cross-entropy cancels the exp in softmax.
- **Its own derivative.** This is what makes the gradient of cross-entropy come out as the clean "predicted minus target" (section 3).

Here, as in nearly all machine learning writing, log is the natural logarithm, base e. The book does not say so explicitly, but its formulas assume it: the perplexity of §3.2 is the exponential of the cross-entropy. A loss computed by hand with $\log_{10}$ will not match anyone else's numbers.

2. Softmax: from logits to probabilities

Book: §3.1, "Cross-entropy". For classification, the model outputs one real number per class, $f(x;w)_y$, called a **logit**. The book interprets it as "the logarithm of a non-normalized probability" and turns the logits into an estimate of the probability of each class:

$$\hat{P}(Y = y \mid X = x) = \frac{\exp f(x;w)_y}{\sum_z \exp f(x;w)_z}$$

Logits can be negative, huge, anything. Softmax turns them into a probability distribution in two steps: exp makes every value positive, and dividing by the sum makes the values add up to 1.

Example. Logits $(2, 1, 0)$. exp gives $(7.39, 2.72, 1)$, which sum to 11.11, so the probabilities are $(0.665, 0.245, 0.090)$.

Only differences between logits matter. Divide the probabilities of two classes and the denominators cancel:

$$\frac{\hat{P}(y)}{\hat{P}(z)} = e^{f_y - f_z}$$

A gap of 1 between two logits means "$e \approx 2.7$ times more likely". Adding the same constant to every logit changes nothing, because it multiplies the numerator and the denominator by the same factor. This is the precise meaning of "logarithm of a *non-normalized* probability": the logits are log-probabilities up to an unknown shared constant.

Why the book prefers "softargmax". The output approximates a one-hot vector pointing at the largest logit; it does not approximate the largest value itself. Multiply the logits $(2, 1, 0)$ by 10 and the output becomes $(0.99995, 0.00005, 0.000000002)$, almost exactly one-hot. Dividing logits by a number before softmax is what the **temperature** setting does when a language model samples text (multiplying by 10 is temperature 0.1): a low temperature sharpens the distribution towards the top choice, a high one flattens it.

3. Cross-entropy

Book: §3.1, "Cross-entropy". The model should give high probability to the true class of each training example (x_n, y_n), so training minimizes

$$\mathcal{L}_{ce}(w) = -\frac{1}{N}\sum_{n=1}^{N} \log \hat{P}(Y = y_n \mid X = x_n)$$

For each example, take the probability the model gave to the correct class, take $-\log$ of it, and average over the N examples. The book's underbrace names the per-example term $L_{ce}(f(x_n; w), y_n)$; the script $\mathcal{L}_{ce}$ is its average.

Why the logarithm. The probability the model assigns to the whole training set is a product of per-example probabilities. The logarithm turns that product into a sum, and the minus sign turns "maximize" into "minimize". Minimizing cross-entropy is maximum likelihood.

How it behaves. $-\log p$ is 0 when $p = 1$ and grows without limit as $p \to 0$. With the logits $(2, 1, 0)$ from section 2: if the true class is the first, the loss is $-\log 0.665 = 0.41$; if it is the third, the loss is $-\log 0.090 = 2.41$. Being confidently wrong is punished hard.

The simplified form. Substitute softmax into the loss and the log cancels the exp in the numerator:

$$L_{ce} = -f_{y_n} + \log \sum_z e^{f_z}$$

The second term is a smooth version of "the largest logit". So the loss reads: the largest logit, minus the true class's logit. It is near zero only when the true class's logit dominates all the others.

Beyond the book

- **The gradient is "predicted minus target".** With respect to each logit, $\partial L / \partial f_z = \hat{P}(z) - \mathbb{1}[z = y_n]$. For the example with the first class true, the gradient is $(0.665 - 1, 0.245, 0.090) = (-0.335, 0.245, 0.090)$: push the true logit up, push the others down in proportion to the probability they took. This clean gradient is one reason softmax and cross-entropy are almost always paired for classification. Section 6.5 derives it with Jacobians.
- **Why "cross-entropy".** The general cross-entropy between a target distribution p and a prediction q is $-\sum_z p(z) \log q(z)$. With a one-hot target, every term but one vanishes, which is why the book's formula has no sum over classes.
- **A sanity baseline.** Guessing uniformly over K classes gives a loss of $\ln K$. For a vocabulary of 50,000 tokens that is $\ln 50\,000 \approx 10.82$. A language model's training loss is exactly this equation with tokens as classes, and its **perplexity** is e^{loss}.

- **$\widehat{P}$ is not a trustworthy confidence.** The hat means "estimate". Modern networks are typically overconfident: a predicted 0.99 does not mean the prediction is right 99 % of the time (Guo et al., "On Calibration of Modern Neural Networks", 2017).
- **A common bug.** Never compute softmax and then take its logarithm as two separate steps: exp overflows for large logits, and the logarithm of a probability that underflowed to 0 is $-\infty$. Libraries compute $-f_y + \log \sum_z e^{f_z}$ directly and stably, and for that reason they take raw logits: PyTorch's `CrossEntropyLoss`, for example. Applying softmax yourself before such a loss computes the wrong quantity and silently degrades training.

4. Contrastive loss (triplet loss)

Book: §3.1, "Contrastive loss". This loss teaches a model to measure distance, when nobody can say what the correct distance is.

The setup. Here $f(x, x'; w)$ takes two inputs and returns a number: how far apart they are. Regression cannot train it, because there is no "true distance" between two photos of faces. All that is known is an ordering: two photos of the same person should be closer than photos of different people. That is what the book means when it says the value to predict is continuous but "the supervision takes the form of ranking constraints". The field is called **metric learning**.

Each training example is a triplet:

- x_a, the **anchor**;
- x_b, a sample of the same class as the anchor (the **positive**);
- x_c, a sample of a different class (the **negative**).

The book minimizes, summed over triplets with $y_a = y_b \neq y_c$,

$$\max\big(\ 0,\ 1 - f(x_a, x_c; w) + f(x_a, x_b; w)\big)$$

Read it as: the negative must be at least 1 farther from the anchor than the positive is, and any shortfall is penalized.

- Positive at distance 0.5, negative at 2.0: $\max(0, 1 - 2 + 0.5) = 0$. The constraint holds; no loss.
- Positive at 0.8, negative at 1.2: $\max(0, 1 - 1.2 + 0.8) = 0.6$. The order is right, but the gap is only 0.4, so the missing 0.6 is penalized.

Why the 1, the margin. Without it, the model could output distance 0 for every pair and get zero loss. The margin forces actual separation.

Why $\max(0, \cdot)$. Once a triplet is satisfied, its loss is exactly zero and so is its gradient: the model stops caring about it. Cross-entropy, by contrast, never reaches zero and always pushes for more confidence.

Beyond the book

- In practice, a network g maps each input to a vector, an **embedding**, and $f(x, x') = \|g(x) - g(x')\|$. What is learned is the embedding; the distance on top of it is a fixed formula.
- The book calls this triplet sum "a contrastive loss". It is usually called **triplet loss**, and in the literature "contrastive loss" on its own often means a different, pairwise formula.
- **The margin only sets the unit** while distances are unconstrained, because the model can rescale them. It becomes a real hyperparameter once embeddings are normalized to unit length, which is the usual practice.
- **Most triplets are useless.** After the start of training, randomly chosen triplets are already satisfied and contribute zero gradient. Deliberately picking hard negatives, called **mining**, is what makes this loss work in practice, and it is where a large part of the engineering effort goes.
- The dominant modern variant, **InfoNCE**, drops the hinge and uses softmax cross-entropy instead: similarities act as logits, and the positive is the "correct class" among many negatives. The book itself uses this form later, for CLIP (§6.6): the similarities between every image and every text in a batch form a matrix, and cross-entropy trains each image to predict its own caption among them, and each caption its own image. Text embedding models for semantic search and retrieval are trained the same way, so the two losses of §3.1 are more closely related than they look.

5. Backpropagation

Book: §3.4. Gradient descent (§3.3) needs, at every step, the gradient of the loss with respect to every parameter. Backpropagation computes all of them, for billions of parameters, in a single backward sweep.

The mechanism

A model is a chain of layers, $x^{(d)} = f^{(d)}(x^{(d-1)}; w_d)$, from the input $x^{(0)} = x$ to the output $x^{(D)}$.

- **Forward pass:** run the input through the layers in order, keeping every intermediate result, the **activations**.
- **Backward pass:** start from the loss and walk back. At each layer, take the gradient arriving from above and do two things with it:
  1. multiply it by the layer's Jacobian with respect to its *parameters*, which gives that layer's gradient $\nabla \ell_{|w_d}$, the one gradient descent needs;
  2. multiply it by the layer's Jacobian with respect to its *input*, which gives the gradient to hand down to the layer below.

A Jacobian is the table of all partial derivatives of a function: how much each output moves when each input moves (section 6 is about them). The whole procedure is the chain rule from calculus, applied layer by layer.

The book's Figure 3.3 draws exactly this: one black arrow forward through each layer, and two products by Jacobians going back.

The tiny example, step by step

A "network" of two layers, each multiplying by one weight, with a squared-error loss:

$$h = w_1 x \qquad y = w_2 h \qquad \ell = (y - 20)^2$$

The values: input $x = 2$, weights $w_1 = 3$, $w_2 = 4$, target 20. Training needs $\partial \ell / \partial w_1$ and $\partial \ell / \partial w_2$: how much the loss changes when each weight is nudged.

Forward pass. The loss is split into two steps, so that each step is one simple operation:

1. $h = w_1 x = 3 \cdot 2 = 6$
2. $y = w_2 h = 4 \cdot 6 = 24$
3. $e = y - 20 = 4$, the error
4. $\ell = e^2 = 16$

Every intermediate value (h, y, e) is kept in memory, because the backward pass needs them.

Backward pass. One rule is used at every step:

$$\text{gradient at a step's input} = \text{gradient at its output} \times \text{local derivative}$$

The local derivative concerns that single operation only, ignoring the rest of the network.

1. **Start at the loss.** $\partial \ell / \partial \ell = 1$, trivially.
2. **Through $\ell = e^2$.** The local derivative of e^2 is $2e = 8$, so $\partial \ell / \partial e = 1 \cdot 8 = 8$.
3. **Through $e = y - 20$.** Subtracting a constant does not change slopes, so the local derivative is 1 and $\partial \ell / \partial y = 8 \cdot 1 = 8$. Meaning: raise y by 0.01 and the loss rises by about 0.08.
4. **Through $y = w_2 h$.** This operation has two inputs, so there are two local derivatives:
   - with respect to w_2 it is $h = 6$, so $\partial \ell / \partial w_2 = 8 \cdot 6 = 48$. That is the first gradient we wanted.
   - with respect to h it is $w_2 = 4$, so $\partial \ell / \partial h = 8 \cdot 4 = 32$. This one goes down to the layer below as its incoming "message".
5. **Through $h = w_1 x$.** Again two inputs:
   - with respect to w_1 it is $x = 2$, so $\partial \ell / \partial w_1 = 32 \cdot 2 = 64$. That is the second gradient.
   - with respect to x it is $w_1 = 3$, giving $32 \cdot 3 = 96$. It is unused here, because x is data, not a parameter. In a deeper network x would be the output of an earlier layer, and 96 would be the message sent to it.

The 32 is computed once and serves everything below it. That reuse is the whole efficiency trick.

Checking by brute force

Nudge each weight by 0.01 and recompute the loss directly:

- $w_2 = 4.01$: $y = 24.06$ and the loss is 16.48, an increase of 0.48. Predicted: $48 \times 0.01 = 0.48$. ✓
- $w_1 = 3.01$: $h = 6.02$, $y = 24.08$ and the loss is 16.65, an increase of 0.65. Predicted: $64 \times 0.01 = 0.64$. ✓

The small mismatch is there because a gradient is exact only for infinitely small nudges. Backpropagation got both numbers from one backward sweep, without re-running the model once per weight. With two weights that saves little; with a billion weights, brute force would need a billion forward passes.

Using the gradients

One step of gradient descent with learning rate 0.001, both weights updated from the same gradients:

$$w_2 = 4 - 0.001 \cdot 48 = 3.952 \qquad w_1 = 3 - 0.001 \cdot 64 = 2.936$$

A new forward pass gives $h = 5.872$, $y = 23.206$, and a loss of 10.28: down from 16 in one step.

What to take from the example:

- **Each step needs only local information:** the incoming message, its own stored inputs, and its own derivative. It never needs to know what the rest of the network looks like, which is why frameworks can automate it.
- **The stored activations were essential:** step 4 needed $h = 6$ and step 5 needed $x = 2$. That is the memory cost of training.
- **A gradient is local.** It says the loss rises 48 units per unit of w_2, *right here*. Take too big a step and that linear prediction breaks down, which is why the learning rate is small.

Beyond the book

- The book calls the gradient computation *and* gradient descent together "backpropagation": "The combination of this computation with the procedure of gradient descent is called backpropagation." Most people use the word for the gradient computation only, and call the update rule (SGD, Adam) the optimizer.
- **No approximation is involved.** Backpropagation computes exactly the derivatives the chain rule gives on paper. Nudging a weight and watching the loss, as in the brute-force check above, is the approximate method.
- **Why backward.** There is one loss and billions of parameters; going backward gives all the gradients in one pass. Section 6.3 shows why.
- **Cost.** The book: most computation is in linear operations, "each requiring one matrix product for the forward pass and two for the products by the Jacobians for the backward pass", so the backward pass costs roughly twice the forward pass. Section 6.5 shows the two products.
- **Memory is the real price.** Inference needs about as much memory as the most demanding single layer; training must keep all the forward activations, so its memory grows with depth (§3.4, "Resource usage"). This, together with a gradient and the optimizer's state (Adam keeps two extra numbers per parameter) for every parameter, is why fine-tuning a model needs far more GPU memory than running it. **Checkpointing** trades some of that memory back for computation: it stores only some activations and recomputes the rest during the backward pass.
- **Vanishing and exploding gradients.** The gradient is a product of many Jacobians, so it can shrink or grow exponentially with depth. Residual connections and normalization layers (book, chapter 4) exist largely to prevent this; section 6.7 explains it.
- Nobody writes it by hand: frameworks record the operations of the forward pass and build the backward pass automatically (**autograd**).

6. Jacobians and backpropagation

The book's description of backpropagation fits in two sentences, both about "multiplying by the Jacobian". The tiny example of section 5 hides what that means, because with one number per layer every Jacobian is a single number. This section works out the general case: what the matrices are, in which order they are multiplied, and why backpropagation never actually builds them.

6.1 What a Jacobian is

A function $f : \mathbb{R}^n \to \mathbb{R}^m$ takes n numbers and returns m. Its **Jacobian** at a point x is the $m \times n$ matrix of all partial derivatives:

$$J_f(x) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}, \qquad (J_f)_{ij} = \frac{\partial f_i}{\partial x_j}$$

One row per output, one column per input. Row i says how output i responds to every input; column j says how every output responds to input j.

What it means: the Jacobian is the best linear approximation of f near x. Nudge the input by a small vector δ and

$$f(x + \delta) \approx f(x) + J_f(x)\,\delta$$

This is the same statement as "derivative = slope", one dimension up. The special cases are familiar:

- $n = m = 1$: the Jacobian is a 1×1 matrix, the ordinary derivative. Every "local derivative" in the tiny example was one of these.
- $m = 1$ (many inputs, one output, like a loss): the Jacobian is a single row, $1 \times n$. Written as a column, that row is the **gradient** ∇f.
- $n = 1$ (one input, many outputs): a single column, the velocity of a curve.

Two Jacobians belong to every layer $x^{(d)} = f^{(d)}(x^{(d-1)}; w_d)$, because the layer has two kinds of input: one with respect to its input activations, $J_{f^{(d)}|x}$, and one with respect to its parameters, $J_{f^{(d)}|w}$. These are the two products in the book's Figure 3.3, and the two local derivatives in steps 4 and 5 of the tiny example.

6.2 The chain rule is a product of Jacobians

If $h = g(x)$ and $y = f(h)$, then

$$J_{f \circ g}(x) = J_f(h)\, J_g(x)$$

The shapes say why it must be so. If x has n components, h has k and y has m, then J_g is $k \times n$, J_f is $m \times k$, and their product is $m \times n$: one row per output of the composition, one column per input. Using the linear approximation twice gives the same result: a nudge δ of x moves h by $J_g\delta$, which moves y by $J_f(J_g\delta)$.

For a whole network followed by a scalar loss $\ell = L(x^{(D)})$, write $J_{f^{(d)}}$ for short for the layer's Jacobian with respect to its input, $J_{f^{(d)}|x}$. Then the gradient of the loss with respect to the network's input, as a row, is

$$\underbrace{J_L}_{1 \times n_D}\; J_{f^{(D)}}\; J_{f^{(D-1)}} \cdots J_{f^{(1)}}$$

where n_D is the width of the last layer. The leftmost factor is a single row, because the loss is one number, so the whole product is a single row too.

Each factor is evaluated at a point: $J_{f^{(d)}}$ at the layer's own input $x^{(d-1)}$, the value it received in the forward pass. That is why the activations must be kept until the backward pass has used them.

For the **parameters** of layer d, the chain runs from the loss down to layer d and ends there, with that layer's Jacobian with respect to its parameters:

$$\nabla \ell^{\top}_{|w_d} = J_L\, J_{f^{(D)}} \cdots J_{f^{(d+1)}}\; J_{f^{(d)}|w}$$

Every layer's product starts with the same prefix $J_L\, J_{f^{(D)}} \cdots$; they differ only in where they stop. Section 6.3 shows why that matters.

The tiny example was this product with every factor 1×1: $\partial \ell / \partial w_1 = 8 \cdot 4 \cdot 2 = 64$, where 8 is the Jacobian of the loss with respect to y, 4 is the Jacobian of $y = w_2 h$ with respect to h, and 2 is the Jacobian of $h = w_1 x$ with respect to its parameter w_1.

6.3 Which end to start from: forward and reverse mode

Matrix multiplication is associative, so the chain can be multiplied in any order. The result is the same; the cost is not.

Take D layers of width n, so that every $J_{f^{(d)}}$ is $n \times n$, followed by the loss.

- **Right to left** (start at the input): $J_{f^{(2)}} J_{f^{(1)}}$ is a matrix times a matrix, n^3 multiplications, and so is every further step. For $D = 10$ layers of width $n = 1000$, that is about 9×10^9 multiplications.
- **Left to right** (start at the loss): $J_L J_{f^{(D)}}$ is a *row vector* times a matrix, n^2 multiplications, and the result is again a row vector. Every step stays a vector times a matrix. For the same network, $10 \times 1000^2 = 10^7$ multiplications: 900 times fewer. In general the saving is a factor of about n, because a matrix-matrix step costs n times as much as a vector-matrix step.

Left to right is **reverse mode**, and it is backpropagation. Each step multiplies the current row vector by one Jacobian: a **vector-Jacobian product**. That is all the book's "multiplying by the Jacobian" means.

Why one backward pass gives every layer's gradient. Going left to right, the running row vector after k steps is exactly the shared prefix $J_L \, J_{f^{(D)}} \cdots J_{f^{(D-k+1)}}$ from section 6.2: the gradient of the loss with respect to that layer's output. At each layer, two products are taken from it. One with the layer's parameter Jacobian gives the gradient for that layer's parameters, which is finished and handed to gradient descent. One with the layer's input Jacobian gives the next running vector, which carries on to the layer below. The prefix is computed once and shared by every layer under it. This is the matrix version of the tiny example's "the 32 is computed once and serves everything below it", and it is the two arrows per layer in the book's Figure 3.3.

Right to left is **forward mode**. In practice it is run as a **Jacobian-vector product**, $J\delta$: push one direction δ through the network together with the forward pass. One such pass gives the derivative of every output along one input direction. It has no shared prefix to reuse for the parameters, so to get the gradient with respect to N parameters it needs N passes, which is "nudge each weight" again, done exactly.

The rule that falls out: **reverse mode wins when there are many inputs and few outputs; forward mode wins in the opposite case.** Training has millions or billions of inputs (the parameters) and one output (the loss), the extreme case for reverse mode. The price is the one section 6.2 named: reverse mode visits the layers in the opposite order from the forward pass, so every activation must be stored until the backward pass reaches its layer.

6.4 Rows or columns: why Figure 3.3 says $J^\top$

The book's text says the gradient is "multiplied by the Jacobian", but its Figure 3.3 shows $J^\top$. Both are right; they use two conventions for the same numbers.

If gradients are rows, as in sections 6.2 and 6.3, a backward step is the gradient times the Jacobian, on the right: $g_{\text{in}} = g_{\text{out}} \, J$. Most texts, the book's figure included, write gradients as **columns**, so that a gradient has the same shape as the vector it belongs to and gradient descent can subtract it directly: $w \leftarrow w - \eta \nabla \ell$. Transposing $g_{\text{in}} = g_{\text{out}} J$ gives

$$\nabla \ell_{|x^{(d-1)}} = J_{f^{(d)}|x}^\top \, \nabla \ell_{|x^{(d)}}$$

So, in column form, a backward step multiplies by the **transposed** Jacobian. If you ever write a backward pass by hand, this is where the transposes come from: a layer that multiplies by W in the forward pass multiplies by $W^\top$ in the backward pass (section 6.5).

From here on, gradients are columns, and ∇_v is short for $\nabla \ell_{|v}$, the gradient of the loss with respect to v.

6.5 Never build the matrix: the Jacobians of real layers

Reverse mode needs only vector-Jacobian products, and for real layers those can be computed without forming the Jacobian at all. Forming it would often be absurd. The three cases below are all that the network

of section 6.6 needs, and most other layers are variations of them. To keep y for the class, as the book does, layer outputs are called z and a.

A linear layer, $z = Wx + b$, with W of shape $m \times n$.

- *With respect to the input:* $\partial z_i / \partial x_j = W_{ij}$, so the Jacobian is W itself. The backward step is $\nabla_x = W^\top \nabla_z$.
- *With respect to the weights:* z has m components and depends on mn weights, so this Jacobian is $m \times mn$. For $m = n = 1000$ that is a billion entries. Almost all are zero: $\partial z_i / \partial W_{jk}$ is x_k when $i = j$ and 0 otherwise, because output i uses only row i of W. Carry out the vector-Jacobian product and the zeros drop out:

$$(\nabla_W)_{jk} = \sum_i (\nabla_z)_i \frac{\partial z_i}{\partial W_{jk}} = (\nabla_z)_j \, x_k, \qquad \text{so} \quad \nabla_W = \nabla_z \, x^\top, \quad \nabla_b = \nabla_z$$

The gradient of a weight matrix is an outer product: the incoming gradient times the stored input. The tiny example's $\partial \ell / \partial w_2 = 8 \cdot h$ was the 1×1 case.

Where the book's "twice as costly" comes from. With a batch, each input is a row of X ($B \times n$), so the forward pass is $Z = XW^\top$ ($B \times m$), the row form of $z = Wx$. In row form the backward steps are the transposes of the column-form ones: $\nabla_x = W^\top \nabla_z$ becomes $\nabla_X = \nabla_Z W$. The weight gradient $\nabla_W = \nabla_Z^\top X$ is the sum over the batch of the per-example outer products $\nabla_z x^\top$. Each of these three matrix products costs about Bmn multiplications: one product forward, two backward, so the backward pass costs about twice the forward pass.

An elementwise activation, $a_i = \sigma(z_i)$, such as ReLU or tanh. Output i depends only on input i, so the Jacobian is **diagonal**: $\mathrm{diag}(\sigma'(z_1), \ldots, \sigma'(z_n))$, an $n \times n$ matrix whose only non-zero entries are on the diagonal. The backward step is just an elementwise product:

$$\nabla_z = \sigma'(z) \odot \nabla_a$$

For ReLU, $\sigma'(z)$ is 1 where $z > 0$ and 0 elsewhere: the gradient passes through the units that were active in the forward pass and is blocked at the others.

Softmax followed by cross-entropy. Softmax $p = \mathrm{softmax}(f)$ is not elementwise: every probability depends on every logit, through the shared denominator. Write $p_i = e^{f_i}/S$ with $S = \sum_k e^{f_k}$, so that $\partial S / \partial f_j = e^{f_j}$. The quotient rule gives

$$\frac{\partial p_i}{\partial f_j} = \frac{\delta_{ij} \, e^{f_i} S - e^{f_i} e^{f_j}}{S^2} = p_i \delta_{ij} - p_i p_j, \qquad J_{\text{softmax}} = \mathrm{diag}(p) - p\,p^\top$$

where δ_{ij} is 1 if $i = j$ and 0 otherwise. At the logits $(2, 1, 0)$ of section 2:

$$J_{\text{softmax}} = \begin{pmatrix} 0.223 & -0.163 & -0.060 \\ -0.163 & 0.185 & -0.022 \\ -0.060 & -0.022 & 0.082 \end{pmatrix}$$

Two things are visible in it. It is **symmetric**, so $J^\top = J$, which the backward step below uses. And every row sums to zero, so $J\mathbf{1} = 0$: nudging all logits by the same amount moves no probability. That is section 2's "adding a constant to all logits changes nothing", now as a statement about a matrix.

The loss $\ell = -\log p_y$, as a function of p, has gradient $-1/p_y$ in component y and 0 elsewhere: $\nabla_p = -e_y / p_y$, where e_y is the one-hot vector of the true class (1 at y, 0 elsewhere). The backward step through softmax is

$$\nabla_f = J^\top \nabla_p = \left(\mathrm{diag}(p) - p\,p^\top \right) \left(-\frac{e_y}{p_y} \right) = -e_y + p = p - e_y$$

The first product picks $-p_y/p_y = -1$ in component y; the second gives $p \cdot (p_y/p_y) = p$. All the divisions by p_y cancel, and what remains is section 3's "predicted minus target". This is why libraries fuse softmax

and cross-entropy into one operation: the combined gradient is simpler and numerically safer than either part on its own.

6.6 A worked example you can follow by hand

A small network, with integer weights so that every number can be checked on paper: 3 inputs, a linear layer to 4 units, ReLU, a linear layer to 3 logits, softmax and cross-entropy. No biases.

$$x = \begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix}, \quad W_1 = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & -1 & 0 \\ -1 & 0 & 1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ -1 & 1 & 1 & 1 \end{pmatrix}$$

The true class is the second one, $y = 2$ (index 1 in the 0-based code below).

Forward pass.

- $z_1 = W_1 x = (3, 1, -1, -2)$
- $a_1 = \mathrm{ReLU}(z_1) = (3, 1, 0, 0)$
- logits $z_2 = W_2 a_1 = (3, 1, -2)$
- $p = \mathrm{softmax}(z_2) = (0.8756, 0.1185, 0.0059)$
- loss $= -\log 0.1185 = 2.13$. The model is confident in the wrong class.

The Jacobians, if we built them. Following section 6.5, as rows and matrices:

- loss with respect to the logits: the row $(p - e_2)^\top = (0.876, -0.882, 0.006)$, shape 1×3;
- logits with respect to a_1: W_2, shape 3×4;
- a_1 with respect to z_1: $\mathrm{diag}(1, 1, 0, 0)$, shape 4×4, because units 3 and 4 were negative and ReLU switched them off;
- z_1 with respect to x: W_1, shape 4×3.

Their product, $(1 \times 3)(3 \times 4)(4 \times 4)(4 \times 3)$, is the 1×3 gradient of the loss with respect to the input.

Backpropagation: the same product, evaluated from the loss end. The steps below write gradients as columns (section 6.4), so each multiplies by a transposed Jacobian. Transposing step 2, for instance, gives $\nabla_{a_1}^\top = \nabla_{z_2}^\top W_2$: the first two factors of the row product above.

1. $\nabla_{z_2} = p - e_2 = (0.876, -0.882, 0.006)$
2. $\nabla_{a_1} = W_2^\top \nabla_{z_2} = (0.870, -0.876, 0.882, -0.876)$
3. $\nabla_{z_1} = \nabla_{a_1} \odot (1, 1, 0, 0) = (0.870, -0.876, 0, 0)$
4. $\nabla_x = W_1^\top \nabla_{z_1} = (0.870, -0.006, -0.876)$

And on the way, at each linear layer, the outer products for the weights:

$$\nabla_{W_2} = \nabla_{z_2} a_1^\top = \begin{pmatrix} 2.627 & 0.876 & 0 & 0 \\ -2.645 & -0.882 & 0 & 0 \\ 0.018 & 0.006 & 0 & 0 \end{pmatrix}, \qquad \nabla_{W_1} = \nabla_{z_1} x^\top = \begin{pmatrix} 0.870 & 1.739 & -0.870 \\ -0.876 & -1.751 & 0.876 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

These were computed from unrounded values; from the three-decimal values above you get 2.628, −2.646, 1.740 and −1.752 in the four places where the last digit differs.

Read the zeros. The last two columns of ∇_{W_2} are zero because units 3 and 4 output 0, so their outgoing weights had no effect on this example. The last two rows of ∇_{W_1} are zero because ReLU blocked the gradient at those units. A unit that is switched off for every training example gets no gradient for its incoming weights, so nothing pushes it back on: a **dead ReLU**.

The check. Building the four Jacobians and multiplying them gives the same ∇_x as the four backward steps:

```
import numpy as np

x  = np.array([1., 2., -1.]); y = 1          # 0-based index of the true class
```

```
W1 = np.array([[1, 1, 0], [0, 1, 1], [1, -1, 0], [-1, 0, 1]], float)
W2 = np.array([[1, 0, 1, 0], [0, 1, 0, 1], [-1, 1, 1, 1]], float)

z1 = W1 @ x; a1 = np.maximum(z1, 0); z2 = W2 @ a1
p  = np.exp(z2 - z2.max()); p /= p.sum()

# The chain rule, literally: build every Jacobian and multiply.
J = (p - np.eye(3)[y])[None, :] @ W2 @ np.diag((z1 > 0).astype(float)) @ W1

# Backpropagation: vector-Jacobian products only, no Jacobian ever built.
g_z2 = p - np.eye(3)[y]
g_a1 = W2.T @ g_z2
g_z1 = g_a1 * (z1 > 0)
g_x  = W1.T @ g_z1
g_W2, g_W1 = np.outer(g_z2, a1), np.outer(g_z1, x)

assert np.allclose(J.ravel(), g_x)
print(g_x.round(3))   # [ 0.87  -0.006 -0.876]
```

The full check script, offered in the download box, also nudges every input and every weight by 10^{-6} and recomputes the loss: the finite differences agree with backpropagation to within 10^{-9}.

6.7 Vanishing and exploding gradients, in Jacobian terms

In column form, the gradient at layer d is the loss gradient multiplied by every transposed Jacobian above it:

$$\nabla_{x^{(d)}} = J_{f^{(d+1)}}^{\top} J_{f^{(d+2)}}^{\top} \cdots J_{f^{(D)}}^{\top} \nabla L$$

Each factor can stretch or shrink the vector. If every layer shrinks it by a factor of 0.9, fifty layers shrink it to $0.9^{50} \approx 0.005$, and the early layers barely learn: the **vanishing gradient**. If every layer stretches it by 1.1, fifty layers multiply it by about 117: an **exploding gradient**.

The book (§3.4, "Vanishing gradient") points out that the cure was to change the models rather than the optimization methods. Residual connections show in Jacobian terms how that works. A residual block computes $y = x + F(x)$, so its Jacobian is $I + J_F$ and its backward step is $g + J_F^{\top} g$. The identity term carries the incoming gradient g down intact, so the product above cannot shrink to nothing merely because the J_F factors are small. Normalization layers help from the other side, by keeping the activations, and with them the derivatives evaluated at them, in a reasonable range.

For the exploding case there is also an optimizer-side patch, which the book mentions as well: **gradient clipping** rescales the gradient when its norm exceeds a threshold, so a single bad step cannot wreck the weights.

6.8 Summary

- A Jacobian is the matrix of partial derivatives of a function with many inputs and many outputs: one row per output, one column per input. It is the best linear approximation of the function near a point.
- The chain rule says that the Jacobian of a composition is the product of the Jacobians, so the gradient of the loss with respect to anything is a product of Jacobians, each evaluated at the activations of the forward pass.
- Backpropagation evaluates those products starting from the loss end. Because the loss is a single number, each step is a vector times a matrix instead of a matrix times a matrix, and the running vector is shared by every layer below it. That is why one backward pass gives every gradient.
- Real layers never build their Jacobians: a linear layer's backward step is a product by $W^{\top}$ plus an outer product for the weights; an elementwise activation's is an elementwise product; softmax with cross-entropy collapses to "predicted minus target".
- The book's "multiply by the Jacobian" and its figure's "$J^{\top}$" are the same operation, written with gradients as rows or as columns.